SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: INTERNET PROGRAMMING **NAME: S SUNDARESAN**

COURSE CODE: CSA4303 **REG NO: 192411245**

COURSE OUTCOME COVERED

CO-1: Develop well-structured, standards-compliant web pages using HTML/XHTML and XML, and apply Internet protocols and HTTP communication mechanisms for web content delivery. **(L2)**

Assessment Tool: Data Interpretation (Medium)

Weightage: 20%

QUESTIONS: **Total Marks: 20**

1: HTTP Response Analysis

A web server returns the following HTTP response header:

```
HTTP/1.1 200 OK
Date: Mon, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: -150
Connection: keep-alive
```

Questions:

1. Identify any incorrect or invalid fields in the header.
2. Analyze the issue with Content-Length and explain its impact.
3. Determine whether the status code matches the response correctness.
4. Suggest corrections for the identified errors.

2: DOM Structure Analysis

Consider the following DOM tree representation:

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
```

```
<div>
<p>HelloWorld
</div>
</body>
</html>
```

Questions:

1. Identify missing or improperly closed elements.
2. Analyze how the browser will interpret this structure.
3. Determine the impact on rendering and layout.
4. Suggest corrections to make the DOM valid.

3: GET vs POST in Login System

A login system uses two different methods:

Method	URL Example	Data Visibility	Security Level
GET	/login?user=admin&pass=1234	Visible in URL	Low
POST	/login	Hidden in body	Moderate

Questions:

1. Interpret the differences in data transmission between GET and POST.
2. Analyze which method is more suitable for login and why.
3. Identify security risks in using GET for authentication.
4. Suggest best practices for secure login implementation.

4: HTML Code Inspection

Analyze the following HTML snippet:

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

Questions:

1. Identify missing closing tags.

2. Analyze issues with the <img> tag.
3. Determine how browsers will handle these errors.
4. Suggest a corrected version of the code.

5: Browser-Server Communication Flow

The following sequence describes a web request:

1. User enters a URL in the browser
2. DNS lookup occurs
3. HTTP request is sent
4. Server processes request
5. Response is returned
6. Browser renders page

Questions:

1. Interpret each step in the communication flow.
2. Identify where delays can occur.
3. Analyze the role of DNS in the process.
4. Suggest optimizations to improve performance.

Code of Conduct:

*I **S SUNDARESAN/192411245** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student: 

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)	Marks
Understanding of Data	25%	Accurately interprets all data patterns, trends, and relationships	Interprets data correctly with minor gaps	Partial understanding; misses key trends	Misinterprets or fails to understand data	24
Analysis	25%	Provides deep analysis with strong logical reasoning and justification	Provides reasonable analysis with some justification	Basic analysis with weak reasoning	No meaningful analysis provided	24
Application of Concepts	20%	Effectively applies relevant concepts/tools to interpret data accurately	Applies concepts with minor errors	Limited application; requires guidance	Unable to apply concepts to data	19
Conclusion	20%	Draws clear, meaningful conclusions with strong insights and implications	Provides valid conclusions with moderate insights	Conclusions are vague or weak	No clear or valid conclusions	19
Clarity	10%	Presents interpretation clearly using proper structure, visuals, and terminology	Generally clear with minor issues	Lacks clarity or proper structure	Poor presentation and difficult to understand	10

1: HTTP Response Analysis

1. Identification of incorrect or invalid fields

Field	Value given	Verdict	Reason
Status line	HTTP/1.1 200 OK	Valid	Correct version, three-digit code and reason phrase.
Date	Mon, 01 Apr 2026 10:00:00 GMT	Format valid, content wrong	The IMF-fixdate format is followed, but 01 April 2026 is a Wednesday, not a Monday. The weekday and date contradict each other.
Content-Type	text/html	Valid (incomplete)	Correct media type. Best practice is to add a charset parameter.
Content-Length	-150	INVALID	The definition is a non-negative decimal integer (1*DIGIT). A minus sign is illegal and a body cannot have negative size.
Connection	keep-alive	Valid	A permitted token; it makes the exact body length critical (see part 2).

Therefore Content-Length is the critical invalid field, and the Date weekday is a secondary inconsistency. In a proper response the Date field is generated by the server clock, never typed manually, so a wrong weekday also suggests a misconfigured or hand-built response.

2. Analysis of the Content-Length issue and its impact

Content-Length states the exact number of body bytes (octets) that follow the blank line. It is the message framing mechanism: the client reads precisely that many bytes and then knows the response has ended. When the connection is persistent (keep-alive), the same TCP connection carries many responses one after another, so the length is the only boundary between them. RFC 9110 requires that an invalid Content-Length value be treated as an unrecoverable error, which means a compliant client must discard the response and close the connection. The effects are:

- **Parse failure:** a client cannot read minus 150 bytes. Browsers and libraries abort the response and show an error such as ERR_INVALID_CONTENT_LENGTH or an exception in the HTTP library. The user sees an error page rather than the HTML document, even though the server reported success.
- **Broken connection reuse:** if a lenient client ignores the value it loses the message boundary. It may wait for data that never comes (hang or timeout), or it may read the start of the next response as part of this body (response desynchronisation), corrupting every later response on the same connection.
- **Intermediary behaviour:** proxies, load balancers and CDNs normally reject the message and return 502 Bad Gateway, or close the connection, so the failure is amplified for many users.
- **Security implications:** disagreement between two systems about where a message ends is the basis of HTTP response splitting, request smuggling and cache poisoning, so an invalid length is treated as a security fault and not only a formatting fault.
- **Performance and reliability:** the failure forces a new TCP/TLS connection and a retry, wasting round trips, and a partial or blank page harms user experience, search indexing and monitoring (health checks show false errors).

3. Does the status code match the correctness of the response?

No. The status code 200 OK declares that the request was understood and fulfilled and that a valid representation accompanies the response. The status line and the remaining headers say the transaction was successful, yet the framing information is invalid, so the message cannot actually be delivered. The status therefore does not agree with the real condition of the response. The code only has meaning if the message itself is well-formed, since a status is useless to a client that rejects the message before it can use it.

Reasoning from the possible causes: (a) if the application merely miscalculated the length, the correct fix is to compute the length properly and keep 200; (b) if the server could not build the response at all (for example a failing script or a buffer underflow that produced the negative number), it should report a server fault with 500 Internal Server Error; (c) if the message passed through a gateway that detected the fault, 502 Bad Gateway is the appropriate status. In short, 200 is acceptable only after the header block is repaired.

Status	Meaning	Suitable here?
200 OK	Request succeeded, valid body follows	Only after Content-Length is corrected
500 Internal Server Error	Server failed while generating the response	Yes, if the response cannot be built correctly
502 Bad Gateway	Intermediary received an invalid response	Yes, when a proxy detects the invalid header

4. Suggested corrections

1. **Content-Length:** compute it automatically as the number of bytes of the body after encoding (for example Content-Length: 150 if the body is 150 bytes). Frameworks and web servers (Apache, Nginx, Node.js, Tomcat) do this, so manual values should be avoided. Note that bytes, not characters, are counted, which matters for UTF-8 text.
2. **Date:** use the correct weekday: Wed, 01 Apr 2026 10:00:00 GMT, generated by the server in GMT.
3. **Alternative framing:** for dynamically generated content whose size is unknown, omit Content-Length and send Transfer-Encoding: chunked. Both headers must never be sent together, because that is a classic smuggling vector.
4. **Content-Type:** add the character set: text/html; charset=UTF-8, so the browser decodes the body correctly.
5. **Operational practice:** validate responses with tools such as curl -i, browser developer tools (Network tab) or Postman, and add automated tests that reject negative or mismatched lengths.

CORRECTED CODE:

HTTP/1.1 200 OK

Date: Wed, 01 Apr 2026 10:00:00 GMT

Content-Type: text/html

Content-Length: 150

Connection: keep-alive

Conclusion

The negative Content-Length is the decisive defect: it breaks message framing, defeats persistent connections and creates security exposure, so the 200 OK status does not truthfully describe the response. After supplying the true byte count and fixing the Date weekday, the response is valid and the status code is consistent with it.

OUTPUT:

1. HTTP Response Analysis – Corrected Header (Output)

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

Output (HTTP Response Header):

```
HTTP/1.1 200 OK
Date: Wed, 01 Apr 2026 10:00:00 GMT
Content-Type: text/html
Content-Length: 150
Connection: keep-alive
```

2: DOM Structure Analysis

The Document Object Model (DOM) is the tree the browser builds from markup. The given representation contains tags with irregular indentation and one element, the , that is never closed.

```
<html>
  <head>
  <title>TestPage</title>
</head>
  <body>
  <div>
  <p>HelloWorld
  </div>
  </body>
  </html>
```

1. Missing or improperly closed elements

- ** is never closed:** there is no . The paragraph is opened and the next end tag in the source is , so the tags overlap (div opens, p opens, div closes) instead of nesting. Proper nesting requires the last opened element to be closed first.
- **<head> and <title>:** <title> is closed and </head> is present, so these are syntactically correct. However the inconsistent indentation (title not indented under head, </head> at a different level) makes the hierarchy hard to read and is poor style.
- **Missing document-level items:** there is no <!DOCTYPE html> declaration, no lang attribute on <html> and no <meta charset> in the head. They are not closing-tag errors but are required for a standards-compliant page.
- **Text without whitespace:** "HelloWorld" is written as one word, which is only a content matter, but it will be displayed exactly that way.

2. How the browser interprets this structure

Browsers follow the HTML5 parsing algorithm, which is designed never to fail. It uses a stack of open elements and repairs mistakes instead of rejecting the page:

1. **Start:** html, head and title are opened and closed normally; the parser then creates the body and opens div and p. Text “HelloWorld” is added as a child text node of the p element.
2. **Encountering :** the open stack is html, body, div, p. The end tag for div is found while p is still open. Because the end tag of a paragraph is optional, the parser implies </p>, pops the paragraph from the stack, and then closes the div.
3. **Encountering </body> and </html>:** these close the remaining elements and the tree is complete. The parser records a parse error but continues.
4. **Missing doctype:** the document is switched to quirks mode, in which the browser imitates old, non-standard behaviour.

Resulting DOM: html → (head → title → “TestPage”) and (body → div → p → “HelloWorld”). Note that this is how an HTML parser repairs it. If the same text were served as XHTML (application/xhtml+xml), the browser uses a strict XML parser instead and would refuse to render the page, showing an “XML Parsing Error: mismatched tag” message. This is the key difference between HTML and XHTML/XML that CO-1 covers: XML requires well-formedness, HTML tolerates errors.

3. Impact on rendering and layout

- **Visible result:** in this small example the user still sees “HelloWorld” as a paragraph inside a div, so the error is invisible, which is exactly why such errors survive testing.
- **Quirks mode:** box model calculation, line-height in tables, image alignment and default font sizes differ from standards mode, so spacing and widths can vary between browsers and from what the designer intended.
- **Larger pages:** where an unclosed paragraph is followed by other block content, the implied closing points may fall in unintended places. CSS selectors such as div > p, margins, floats, flex or grid containers then apply to the wrong nodes and the layout shifts.
- **Scripting:** JavaScript methods such as parentElement, children and querySelector may return different results from those the developer expects, creating hard-to-find bugs.
- **Accessibility, SEO and maintenance:** assistive technology depends on a clean tree; validators (W3C) report errors; search engines may parse the structure inconsistently; and later edits by other developers are more error-prone.

Aspect	Effect of the error
Rendering of text	Appears normally (parser auto-closes p)
Rendering mode	Quirks mode due to missing doctype
Layout stability	Risk of unintended closing points and inconsistent spacing
XHTML/XML context	Fatal well-formedness error; page not displayed
Validation	Fails W3C validator (parse errors)

4. Corrections to make the DOM valid

Close every element in the reverse order of opening, add the doctype and the document metadata, use consistent indentation, and separate the words:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <title>Test Page</title>
  </head>
  <body>
    <div>
      <p>Hello World</p>
    </div>
  </body>
</html>
```

For XHTML, the same document would additionally need the xmlns attribute on the root element (xmlns="http://www.w3.org/1999/xhtml"), lower-case element names, quoted attribute values and self-closing syntax for empty elements.

CORRECTED CODE:

<!DOCTYPE html>

<html>

<head>

<title>TestPage</title>

</head>

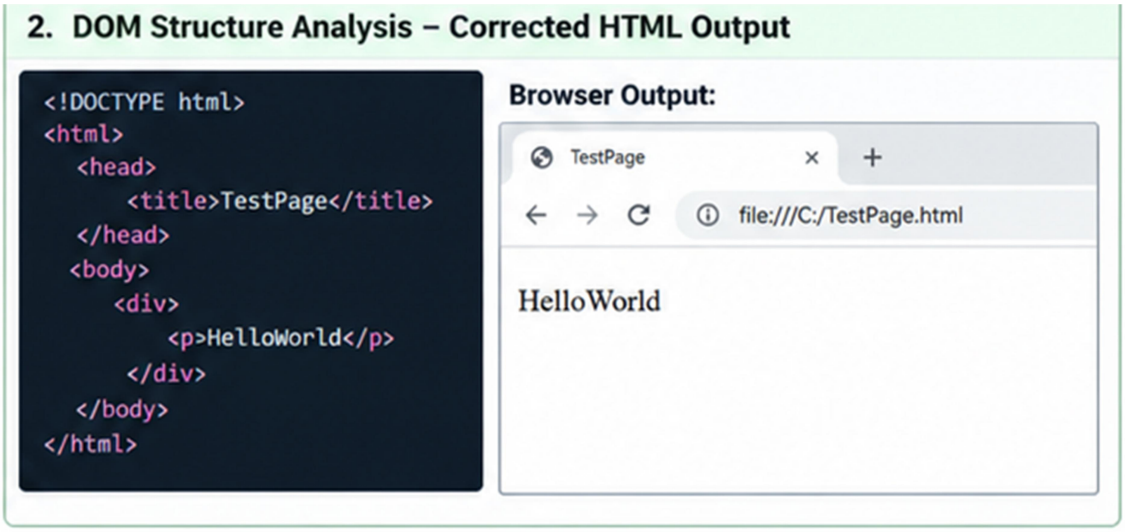
<body>

```
    <div>
        <p>HelloWorld</p>
    </div>
</body>
</html>
```

Conclusion

The one structural fault is the unclosed <p>, aggravated by the missing doctype. HTML parsers silently repair it, but relying on that repair gives unpredictable layout and fails validation, and a strict XML parser would reject it outright. The corrected version is well-formed, valid and renders the same everywhere.

OUTPUT:



3: GET vs POST in Login System

1. Differences in data transmission between GET and POST

Both methods are defined in RFC 9110. GET asks the server to retrieve a representation of a resource; POST asks the server to process data enclosed in the request. They differ in where the data travels and in how browsers and servers treat it:

Feature	GET	POST
Location of data	In the URL as a query string, in the request line	In the message body, after the headers
Example request	GET /login?user=admin&pass=1234 HTTP/1.1	POST /login HTTP/1.1 with body user=admin&pass=1234
Visibility	Shown in address bar, history, logs	Not shown in the address bar or history
Size limit	Limited by URL length (about 2,000 characters in practice)	No practical limit (server configured)
Data types	Text, percent-encoded	Text, binary, files (multipart/form-data)
Caching / bookmarking	Cacheable and bookmarkable	Not cached or bookmarked by default
Semantics	Safe and idempotent (read only)	Not idempotent, changes server state
Back/refresh	Harmless to repeat	Browser warns before re-submitting

The table rates the visibility column as “Visible in URL” versus “Hidden in body”. “Hidden” must be understood carefully: the data is merely not displayed in the URL. It is still plain text on the network, so only HTTPS/TLS actually protects it. This explains why POST is graded only “Moderate” rather than “High”.

2. Which method is more suitable for login, and why

POST is the suitable method, for these reasons:

- **Sensitive data:** a password must not be part of a URL. URLs are designed to be shared, copied, logged and stored, while bodies are not.

- **Correct semantics:** logging in creates a session on the server, i.e. it changes state. GET must be safe and repeatable, so using it to authenticate violates the protocol design.
- **Leak surfaces avoided:** with POST the credentials do not appear in the browser history, bookmarks, server access logs, proxy logs, the Referer header or shared screenshots.
- **Fewer automatic triggers:** GET can be fired by links, images, browser prefetching and crawlers; a login by GET could be triggered unintentionally or maliciously.
- **Flexibility:** POST allows larger and structured data (tokens, CAPTCHA responses, multi-factor codes) without URL limits.

It must be stressed that POST alone does not secure a login. It is the correct foundation that must be combined with HTTPS and the practices in part 4.

3. Security risks of using GET for authentication

1. **Shoulder surfing:** the password is displayed in plain sight in the address bar.
2. **Browser history and bookmarks:** the whole URL is stored locally, so another user of the device can recover the credentials.
3. **Server, proxy and CDN logs:** access logs record request URLs, leaving passwords in plain text in files that many administrators and tools can read.
4. **Referer leakage:** if the next page links to an external site, the browser may send the previous URL, including the query string, to that third party.
5. **Caching and replay:** URLs can be cached by browsers or proxies, replayed by anyone who obtains the link and indexed by crawlers or analytic tools.
6. **CSRF and unintended execution:** an attacker can embed a login URL in an image tag or link, forcing the victim's browser to log in to the attacker's account (login CSRF).
7. **Network exposure:** without HTTPS every method is sniffable, but with GET the credentials are also exposed in places where HTTPS does not help (history, logs, Referer).
8. **Length and encoding problems:** special characters in passwords can be mangled or truncated by URL encoding and length limits, causing failed logins and workarounds that weaken security.

4. Best practices for implementing a secure login

- **Use POST over HTTPS only:** enforce TLS 1.2 or 1.3, redirect HTTP to HTTPS and enable HSTS so browsers never send credentials in clear text.

- **Store passwords safely:** never in plain text. Hash with a slow, salted algorithm such as bcrypt, scrypt, Argon2 or PBKDF2, and consider a server-side pepper.
- **Secure sessions:** generate long random session identifiers, set cookies with HttpOnly, Secure and SameSite attributes, regenerate the identifier after login, set expiry and invalidate it on logout.
- **Prevent injection and forgery:** use parameterised queries or prepared statements against SQL injection, escape output against XSS and include an anti-CSRF token in the login form.
- **Limit brute-force attempts:** rate limiting, temporary lockout, CAPTCHA after repeated failures and monitoring of suspicious IP addresses.
- **Strengthen authentication:** multi-factor authentication, strong password policy, and avoidance of default accounts such as admin/1234 shown in the example.
- **Safe error handling:** show one generic message ("Invalid username or password") so attackers cannot learn which part was wrong, and never log passwords.
- **Form hygiene:** use autocomplete attributes appropriately, input type="password" to mask entry, and server-side validation of all fields.

```
<form action="/login" method="POST">
  <input type="text" name="user" required>
  <input type="password" name="pass" required>
  <input type="hidden" name="csrf_token" value="...">
  <button type="submit">Login</button>
</form>
```

CORRECTED CODE:

```
<!DOCTYPE html>
<html>
<head>
   <title>Login Page</title>
</head>
<body>
   <h2>User Login</h2>

   <form action="/login" method="post">
```

```
    <label for="user">Username:</label>
    <input type="text" id="user" name="user" required>
    <br><br>

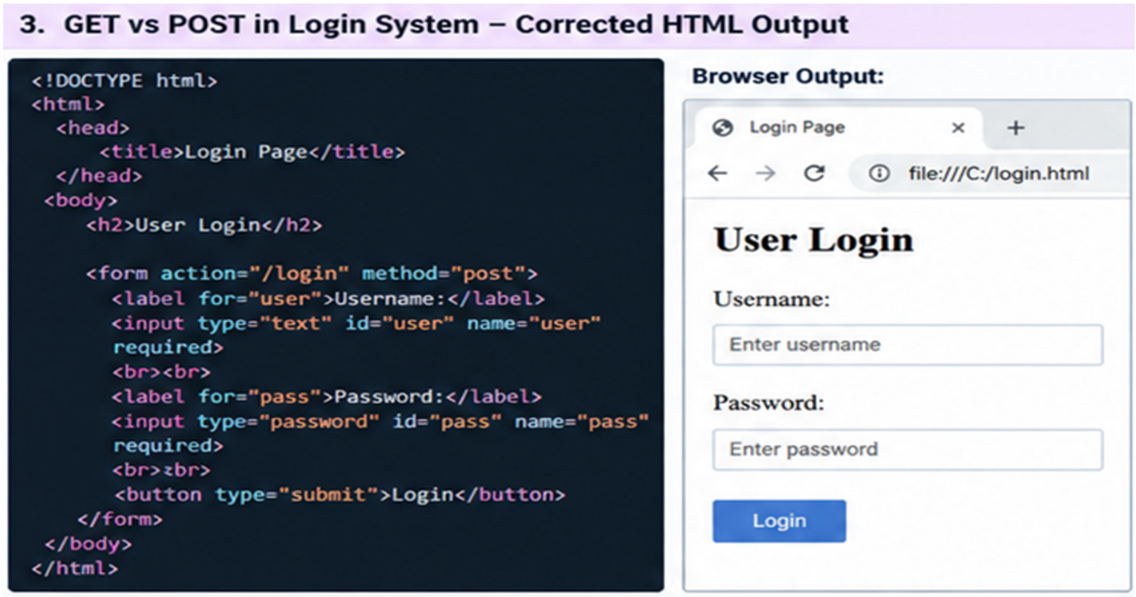
    <label for="pass">Password:</label>
    <input type="password" id="pass" name="pass" required>
    <br><br>

    <button type="submit">Login</button>
  </form>
</body>
</html>
```

Conclusion

GET exposes credentials in the URL, history, logs and Referer header and breaks the protocol's semantics, so it is unsuitable for authentication. POST, delivered over HTTPS and combined with hashing, secure cookies, CSRF protection and rate limiting, is the correct and secure design.

OUTPUT:



4: HTML Code Inspection

```
<!DOCTYPE html>
<html>
<head>
<title>Sample</title>
</head>
<body>
<h1>Welcome
<p>This is a paragraph
<img src="image.jpg">
</body>
</html>
```

1. Missing closing tags

- **</h1> is missing:** the heading text "Welcome" is never closed. Heading elements require an end tag, and omitting it is a parse error.
- ** is missing:** the paragraph is not closed. HTML5 allows the end tag of p to be omitted in certain situations (before another block element or at the end of its parent), so the parser tolerates it, but explicit closing is the standards-compliant and recommended practice.
- **Elements that are correct:** html, head, title and body are closed. The <img> element is a void element, which has no content and therefore no closing tag; writing </img> would be wrong. <!DOCTYPE html> also takes no closing tag.

Element	Closed?	Remark
<h1>	No	Parse error; must add </h1>
	No	Tolerated by HTML5 but should be closed
<img>	Not needed	Void element
<body>, <html>, <head>, <title>	Yes	Correct

2. Issues with the <img> tag

- **Missing alt attribute:** alt is mandatory in HTML5 for a valid document. Without it, screen-reader users cannot know what the image shows, nothing is displayed to describe the image if it fails to load or the user has images disabled, and search engines lose context for image indexing.
- **No width and height:** the browser cannot reserve space until the file is downloaded, so the page content jumps when the image appears (layout shift), which harms usability and Core Web Vitals.
- **Incorrect placement:** because the paragraph and heading are unclosed, the image becomes nested inside them (see part 3). A decorative or block image is better placed outside the paragraph, or wrapped in <figure> with a <figcaption>.
- **File path risk:** src="image.jpg" is a relative path, so the file must be in the same folder as the page; otherwise the browser shows a broken-image icon. The file name is also not descriptive (poor for SEO).
- **Performance and responsiveness:** no loading="lazy" attribute or responsive attributes (srcset, sizes, CSS max-width) are present, so large images may slow the page on mobile devices.

3. How browsers handle these errors

The HTML5 parser does not stop on errors. Following its rules, the browser builds the DOM as follows:

1. **<h1>Welcome:** h1 is opened and the text node “Welcome” is added.
2. **<p>:** a start tag for p does not close an open h1 (it only closes another open p). The paragraph is therefore created as a child of the h1, not as a sibling.
3. **<img>:** the image is inserted as the next child inside the paragraph, and the void element is closed immediately.
4. **</body>:** the parser closes all still-open elements, p and h1, and records a parse error because h1 was not closed. The paragraph’s missing end tag is tolerated at this point.

Resulting tree: body → h1 → [“Welcome”, p → [“This is a paragraph”, img]]. The practical consequences are:

- **Wrong styling:** the paragraph text and the image are inside the heading, so they inherit the large, bold heading font and its margins. The paragraph looks like part of the title.

- **Wrong semantics:** screen readers and search engines treat the whole content as heading text, which damages the document outline and accessibility.
- **CSS and script problems:** selectors such as h1 + p will not match (p is not a sibling), and scripts that read h1.textContent will return the paragraph text too.
- **Missing image fallback:** with no alt attribute and a possibly wrong path, a broken image leaves an empty space with no explanation.

4. Corrected version of the code

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-
scale=1.0">
  <title>Sample</title>
</head>
<body>
  <h1>Welcome</h1>
  <p>This is a paragraph</p>
  <img src="image.jpg" alt="Sample illustration" width="300"
height="200">
</body>
</html>
```

Improvements made: h1 and p are properly closed and become siblings; the image follows the paragraph rather than being nested in it; alt, width and height are added; the lang attribute, character set and viewport meta tag make the page accessible and mobile-friendly. The result validates without errors in the W3C validator.

Conclusion

The snippet shows how forgiving HTML parsers hide errors: the page still displays, but the missing </h1> silently nests the paragraph and image inside the heading and the image lacks alt text. Writing well-formed, validated markup removes these risks and gives predictable, accessible output in every browser.

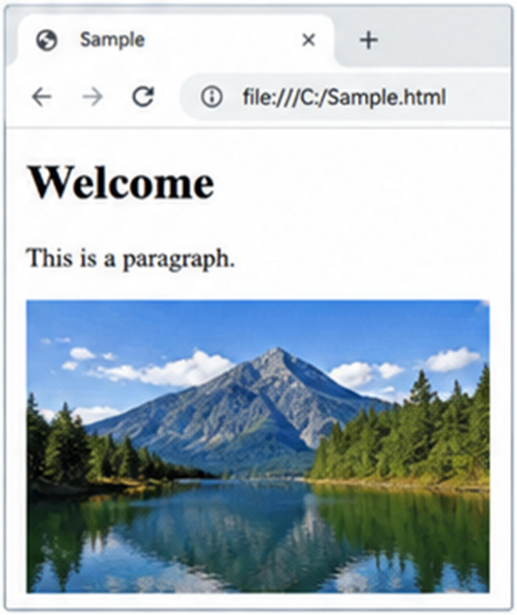
OUTPUT:

4. HTML Code Inspection – Corrected HTML Output

```
<!DOCTYPE html>
<html lang="en">
  <head>
      <meta charset="UTF-8">
      <meta name="viewport"
      content="width=device-width,
       initial-scale=1.0">
      <title>Sample</title>
</head>
 <body>
     <h1>Welcome</h1>
     <p>This is a paragraph.</p>

     <img src="image.jpg"
          alt="Description of the image"
          width="400"
          height="300">
  </body>
</html>
```

Browser Output:



5: Browser–Server Communication Flow

1. Interpretation of each step

Step	What happens
1. User enters a URL	The browser parses the URL into scheme (https), host (www.example.com), port (443), path and query. It checks the HSTS list, the browser cache and any service worker. If a fresh cached copy exists the network is not used at all.
2. DNS lookup	The host name is resolved to an IP address. The browser cache, operating-system cache and hosts file are checked first, then the recursive resolver of the ISP, which if needed queries the root servers, the TLD servers (.com) and the authoritative name server for the domain.
3. HTTP request is sent	A TCP connection is opened (three-way handshake: SYN, SYN-ACK, ACK) and, for HTTPS, a TLS handshake negotiates encryption. The browser then sends a request such as GET /index.html HTTP/1.1 with headers like Host, User-Agent, Accept and cookies.
4. Server processes the request	The web server (Apache, Nginx, IIS) routes the request, applies security rules, runs application code (PHP, Node.js, Java), queries databases or caches and generates the response content.
5. Response is returned	The server sends a status line (200 OK), headers (Content-Type, Content-Length, Cache-Control, Set-Cookie) and the body (HTML). Compression such as gzip or Brotli may be applied.
6. Browser renders the page	The browser parses HTML to build the DOM, loads CSS to build the CSSOM, combines them into a render tree, performs layout and paint, and runs JavaScript. Linked resources (CSS, JS, images, fonts) trigger more requests, repeating steps 3 to 5.

2. Where delays can occur

Stage	Cause of delay	Typical effect
DNS lookup	Uncached lookup passing through several name servers; slow resolver; long CNAME chains	Tens to hundreds of milliseconds before any connection starts
Connection setup	TCP handshake plus TLS handshake need several round trips; long physical distance	Adds 1–3 round-trip times of latency

Stage	Cause of delay	Typical effect
Network transfer	Congestion, packet loss, low bandwidth, large uncompressed responses	Slow download of the page and assets
Server processing	Slow database queries, heavy computation, overloaded server, cold start	High Time To First Byte (TTFB)
Rendering	Large render-blocking CSS/JS, unoptimised images, many extra requests, complex layout	Blank or incomplete page for a long time
Redirects	Chains such as http → https → www	Each redirect repeats DNS/connection work

The most significant delays usually occur in DNS (first visit), connection setup (long distances) and server processing (dynamic pages), while on repeat visits rendering and the number of resources dominate.

3. Role of DNS in the process

The Domain Name System is a distributed, hierarchical database that works like the phone book of the Internet. Computers route packets using IP addresses, but humans use names, so DNS translates a domain name into an IP address (A record for IPv4, AAAA for IPv6) before step 3 can happen. Without a result from DNS no connection can be made, even if the server is perfectly healthy. DNS also supports:

- **Aliases and records:** CNAME for aliases, MX for mail servers, TXT for verification and e-mail security (SPF, DKIM).
- **Load balancing and geographic routing:** answers can return different IPs depending on the user's location or server load (used by CDNs), and several addresses give redundancy.
- **Flexibility:** an administrator can move a site to a new server by changing the record, without users changing the name they type.
- **Caching:** each record has a TTL (time to live) so answers are reused at the browser, OS and resolver levels, reducing lookups and delay.
- **Security:** DNS is vulnerable to spoofing and cache poisoning; DNSSEC authenticates answers, and DNS over HTTPS/TLS encrypts the queries.

4. Suggested optimisations to improve performance

Area	Optimisation
DNS	Use fast, reliable DNS providers; set sensible TTLs; add dns-prefetch and preconnect hints for third-party domains; reduce the number of distinct hostnames.
Connection	Use HTTP/2 or HTTP/3 (multiplexing, header compression), TLS 1.3 and session resumption, keep-alive connections, and avoid redirect chains.
Network	Serve static assets from a CDN near the user; compress text with Gzip or Brotli; enable caching headers (Cache-Control, ETag).
Server	Cache query results and pages (Redis, Memcached), index and tune database queries, use load balancing and horizontal scaling, and keep application code efficient.
Browser rendering	Minify and bundle CSS/JS, inline critical CSS, load scripts with defer or async, compress images (WebP, AVIF), lazy-load off-screen images and set width/height to prevent layout shift.
Measurement	Use browser DevTools, Lighthouse and WebPageTest to measure TTFB, First Contentful Paint and Largest Contentful Paint, then fix the slowest stage first.

Conclusion

A web page load is a chain: URL parsing, DNS, connection, request, server processing, response and rendering. Total delay is the sum of all stages, and DNS is the critical first dependency. By caching and prefetching DNS, using modern protocols and CDNs, tuning the server and optimising front-end resources, the overall load time can be reduced substantially.

CORRECTED CODE:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>Browser-Server Communication</title>
</head>
<body>
```

```
    <h1>Browser-Server Communication Flow</h1>

    <ol>
        <li>User enters a URL in the browser.</li>
        <li>DNS resolves the domain name to an IP address.</li>
        <li>The browser establishes a connection and sends an HTTP request.</li>
        <li>The server processes the request.</li>
        <li>The server returns an HTTP response.</li>
        <li>The browser processes and renders the webpage.</li>
    </ol>
</body>
</html>
```

OUTPUT:

